



Exceptional service in the national interest

Simulation Tool for Asynchronous Cortical Streams (STACS)

Overview and Tutorial

Felix Wang

NICE 2026 Tutorials

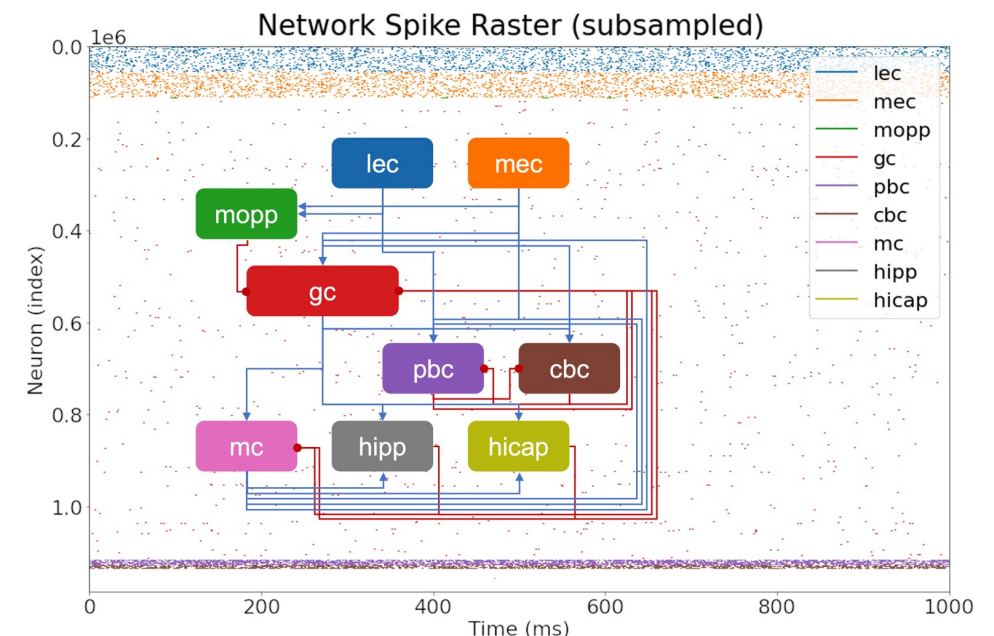


<https://github.com/sandialabs/STACS>

Simulation Tool for Asynchronous Cortical Streams

- STACS is a large-scale spiking neural network simulator built on top of the Charm++ parallel programming framework (runtime with parallel objects)
 - Network models are represented as a directed graph, and neuron (time-driven) and synapse (event-driven) dynamics are formulated as stochastic differential equations
 - Parallelization is supported by partition-based data structures, and serialized through SNN extensions to the distributed compressed sparse row (SNN-dCSR) data format
 - STACS provides scalable methods for graph generation, (re)partitioning, simulation, and serialization. It supports the neuromorphic computing toolchain as an interface between software frameworks and hardware platforms
- Available at: <https://github.com/sandialabs/STACS>

Simulation of a biologically inspired spiking neural network of about 12M neurons and 70B synapses



The graphic features a central dark blue diamond with the text "STACS INSTALLATION" in white. This diamond is surrounded by a white border and is flanked by two diagonal lines of colorful segments (cyan, orange, green, red, purple) extending from the corners towards the center.

STACS INSTALLATION

Charm++ environment installation

- The hardest part of installing STACS is setting up the Charm++ environment
 - Charm++ is actually quite portable, and can run on basically any machine
 - However, it is a C++ based framework, without a package manager like Python
- Setting up the dependencies/environment (e.g. Linux Ubuntu)
 - `sudo apt-get install build-essential gfortran cmake`
 - `sudo apt-get install libyaml-cpp-dev`
 - `sudo apt-get install mpich`
- Setting up Charm++
 - Clone repo from GitHub: <https://github.com/UIUC-PPL/charm>
 - Alternatively, get version 8.0.0 (available as tarball, zip)
 - `./build charm++ mpi-linux-x86_64 -j8`
 - Set up associated environment variables (e.g. in `.bashrc`)
 - `export PATH=/path/to/charm/bin:$PATH`

STACS installation

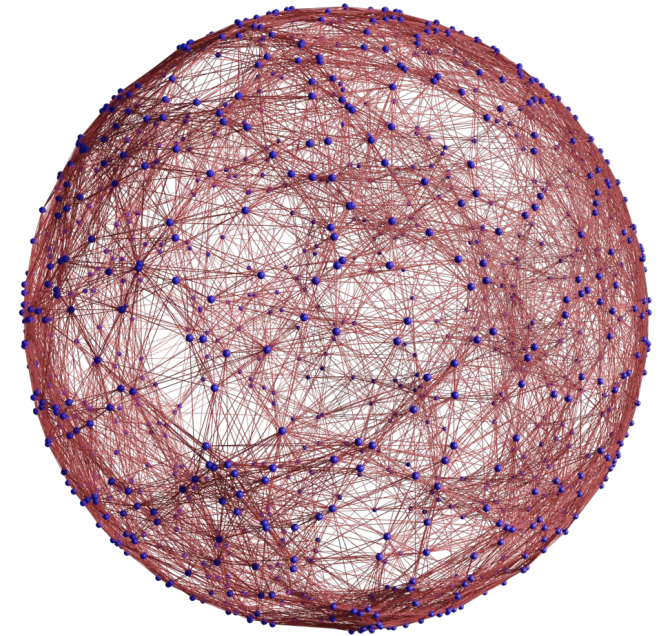
- After Charm++ is installed, STACS uses a trivial Makefile to compile, and we can also install various Python packages for a more user-friendly front-end interface
- Setting up STACS
 - Clone repo from GitHub: <https://github.com/sandialabs/STACS>
 - `make -j8`
- Setting up various Python packages
 - Use your typical python environment (e.g. Anaconda)
 - `pip install pyyaml networkx`
- Setting up Sango (for a more user-friendly front-end interface)
 - Clone repo from GitHub: <https://github.com/sandialabs/Sango>
 - `python -m pip install -e .`

The graphic features a central dark blue diamond with the text 'STACS OVERVIEW' in white. This diamond is surrounded by a white border and is flanked by two diagonal lines of colored segments (cyan, orange, green, red, purple) that extend towards the corners of the frame.

STACS OVERVIEW

Representing a network model in STACS

- Network models are represented as a directed graph, and neuron and synapse dynamics are formulated as stochastic differential equations
 - Network: $G(V, E): V = \{v_1, v_2, \dots, v_n\}$ $E = \{e_1, e_2, \dots, e_m\}$
 where $e_k: v_i \rightarrow v_j$ $i, j \in 1, \dots, n$ and $k \in 1, \dots, m$
 - State dynamics: $dx = f(x)dt + \sum_{i=1}^n g_i(x)dN_i$
 - Drift process evolves state continuously in time
 - Diffuse processes map to network events (e.g. spikes)



Visualization of a 6000 neuron network over the surface of a sphere

$$v' = 0.04v^2 + 5v + 140 - u + I$$

$$u' = a(bv - u)$$

$$\text{if } v \geq v_{th}, \text{ then } \begin{cases} v \leftarrow c \\ u \leftarrow u + d \end{cases}$$

Izhikevich neuron equations (top)

$$\tau_+ \frac{dx_i}{dt} = -x_i + a_+(x_i) \sum_m \delta(t - t_i^m)$$

$$\tau_- \frac{dy_j}{dt} = -y_j + a_-(y_j) \sum_n \delta(t - t_j^n)$$

$$\frac{dw_{ij}}{dt} = A_{LTP}(w_{ij})x_i(t) \sum_n \delta(t - t_j^n)$$

$$- A_{LTD}(w_{ij})y_j(t) \sum_m \delta(t - t_i^m)$$

Trace-based STDP synaptic weight updates (right)

Representing network connectivity as a sparse matrix

Sparse Matrix

	0	1	2	3	4	5	6	7	8	9
I	0				a			b		
	1		c			d				
	2						e		f	
	3			g				h		i
II	4	j			k		l		m	n
	5		o	p				q		
	6	r				s				
III	7			t		u				
	8					v				w
	9	x	y						z	

Compressed Sparse Row

Row: [0, 2, 4, 6, 9, 14, 17, 19, 21, 23, 26]

Col: [4, 7, 2, 5, 6, 8, 2, 7, 9, 0, 3, 5, 7, 9, 1, 2, 6, 0, 4, 3, 5, 5, 9, 0, 1, 8]

Val: [a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z]

Distributed Compressed Sparse Row

Part : [0, 4, 7, 10]

Row I : [0, 2, 4, 6, 9]

Row II : [0, 5, 8, 10]

Row III: [0, 2, 4, 7]

Col I : [4, 7, 2, 5, 6, 8, 2, 7, 9]

Col II : [0, 3, 5, 7, 9, 1, 2, 6, 0, 4]

Col III: [3, 5, 5, 9, 0, 1, 8]

Val I : [a, b, c, d, e, f, g, h, i]

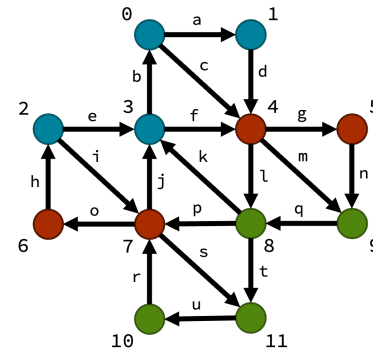
Val II : [j, k, l, m, n, o, p, q, r, s]

Val III: [t, u, v, w, x, y, z]

- Numbered values correspond to row and column coordinates
- Lettered values correspond to non-zero entries
- Roman numerals correspond to partitions
- Entries within a row are uniquely colored for ease of correlation within their corresponding arrays

Extending the dCSR format for spiking neural networks

- The SNN extension to dCSR straightforwardly assigns neuron and synapse state tuples to the matrix values
- As a text-based file format, we may split the index and value information into separate, but aligned files
- Each partition provides “self-contained” state information (except for external events)



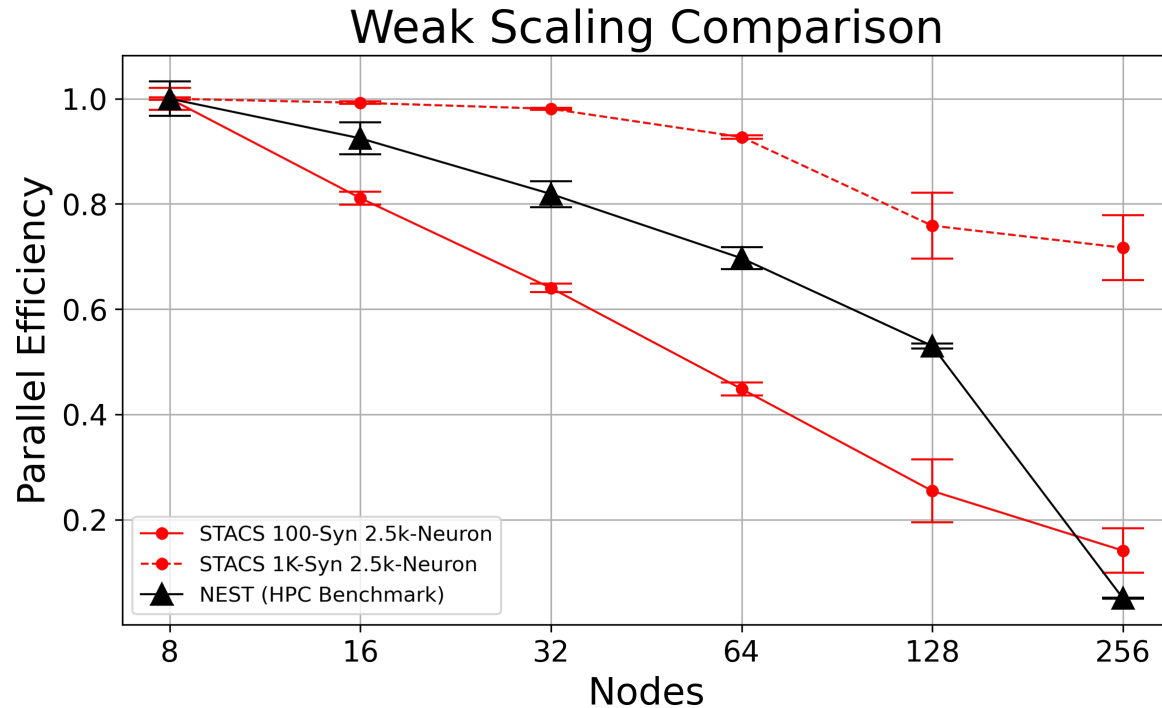
dist	adjcy.0	adjcy.1	adjcy.2
0 0	1 3 4	0 1 3 5 8 9	3 4 7 9 11
4 13	0 4	4 9	4 5 8
8 29	3 6 7	2 7	7 11
12 42	0 2 4 7 8	2 3 6 8 10 11	7 8 10
state.0	state.1	state.2	
0 $\emptyset$ b $\emptyset$	4 c d f $\emptyset$ $\emptyset$ $\emptyset$	8 $\emptyset$ l $\emptyset$ q $\emptyset$	
1 a $\emptyset$	5 g $\emptyset$	9 m n $\emptyset$	
2 $\emptyset$ h $\emptyset$	6 $\emptyset$ o	10 $\emptyset$ u	
3 $\emptyset$ e $\emptyset$ j k	7 i $\emptyset$ $\emptyset$ p r $\emptyset$	11 s t $\emptyset$	

Example 3-way partitioning of a simple network with 12 vertices and 21 directed edges using the SNN-dCSR format. Directed edges with associated state are bolded.

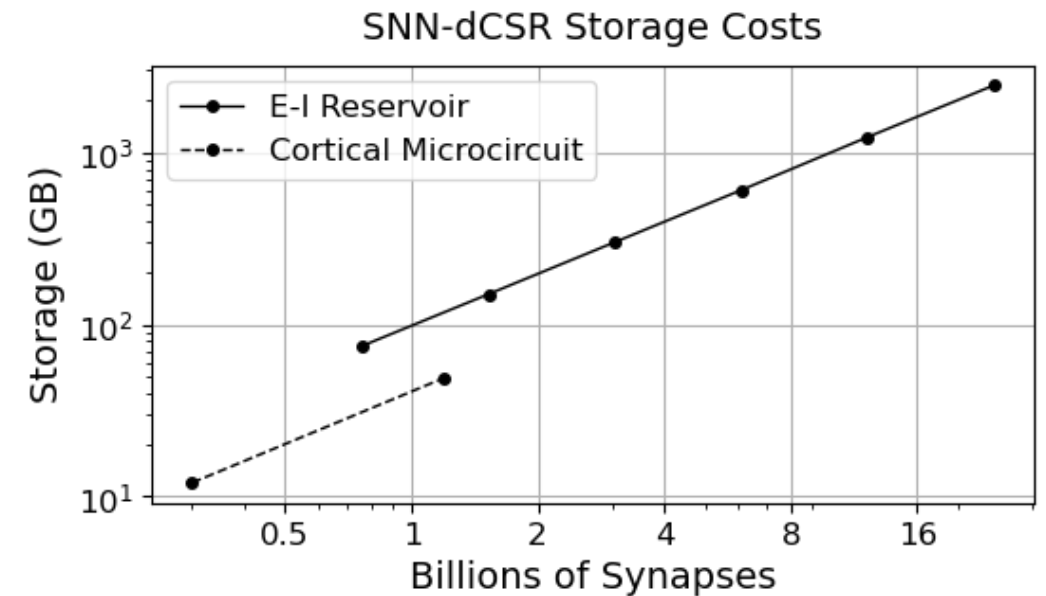
- ‘Part’ information in `dist` file
- ‘Row’ information implicit in `adjcy` files (adjacency lists)
- ‘Col’ information explicit in `adjcy`
- ‘Val’ information in `state` files

Built for scalability in high-performance computing

- Scaling experiments with a spatially parameterized E-I reservoir model showed competitive parallel efficiency in simulation times, and close to linear scaling in network build and partitioning times and storage costs



For this system, 256 nodes was 10240 CPU cores / partitions
(for comparison, a Loihi 2 32-chip board has 3840 neurocores)

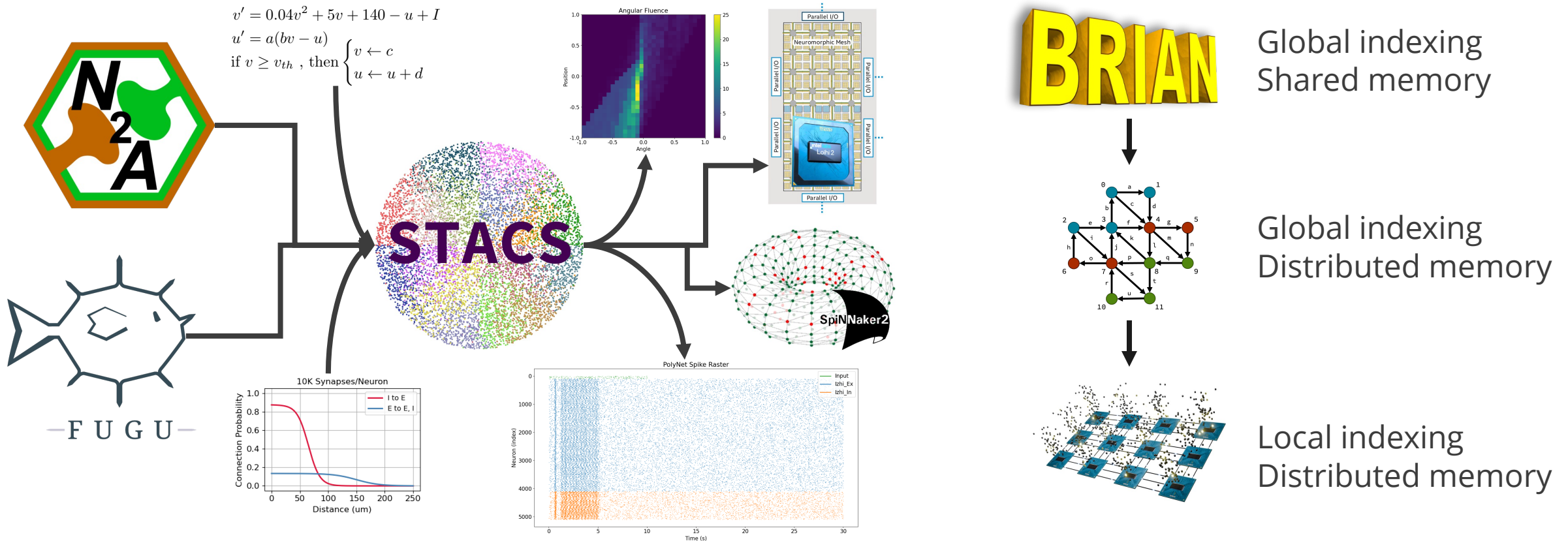


Storage costs are largely dominated by synaptic state, with minimal overhead from additional metadata files

Wang et al., "Scaling neural simulations in STACS", Neuromorphic Computing and Engineering, vol. 4, pp. 024002, 2024

Designed with interoperability in representations

- STACS supports interoperability between and across various tools and platforms along the neuromorphic software and hardware stack



Where STACS sits within the neuromorphic ecosystem

- Description Languages
 - Sango, PyNN, NeuroML, etc.
- Software Frameworks
 - Fugu, N2A, Lava, snnTorch, etc.
- Network Simulators
 - NEST, Brian, GeNN, CARLsim, etc.
- Data Formats
 - NIR, SONATA, NetworkX, etc.
- Hardware Platforms
 - Loihi, SpiNNaker, BrainScaleS, etc.



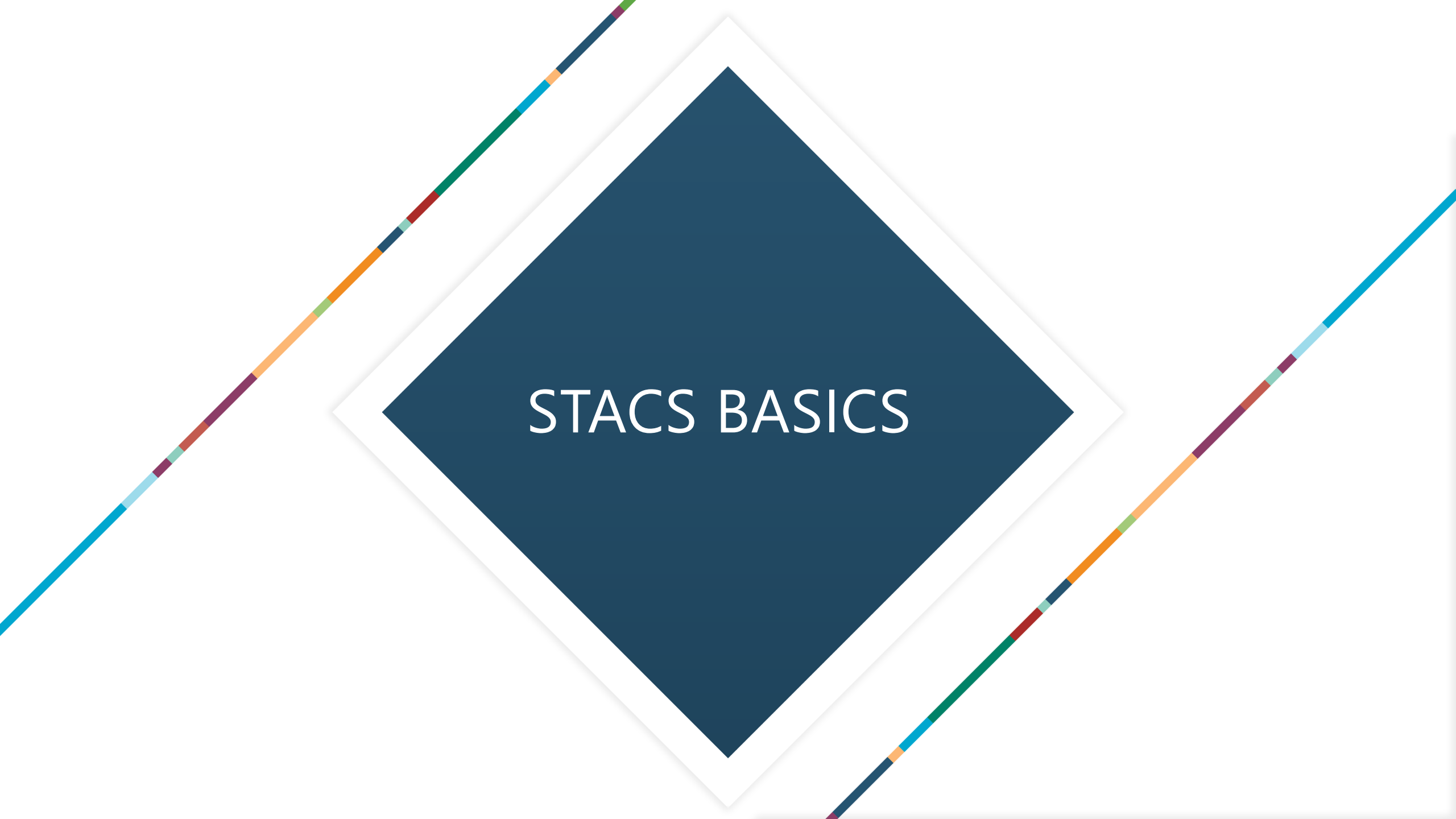
- STACS is able to interoperate with software frameworks such as N2A or Fugu through:
- Translating between higher-level network description languages (N2A → YAML)
 - As a simulation backend for instantiated networks (Sango → SNN-dCSR)



STACS is primarily a spiking neural network simulator



- The partition-based SNN-dCSR data format supports external tool interoperability:
- Such as graph partitioners (e.g. ParMETIS), and network analysis (e.g. GNATFinder)
 - It also provides a path toward mapping instantiated networks to scalable neuromorphic hardware platforms

The slide features a central dark blue diamond with a white border, containing the text "STACS BASICS". Two diagonal lines, composed of small colored segments (cyan, orange, green, red, purple), cross the diamond from the corners.

STACS BASICS

The structure of a STACS simulation experiment

- When developing networks, there are three YAML files that need to be defined:
 - Network simulation config `.yaml` sets up information related to how the network will be processed by the simulator
 - Network graph model `.graph` defines the structure of the vertices and edges that compose the network
 - Network substrate model `.model` defines the substrate dynamics used by the vertices and edges
- As a quick start example, we may copy over an existing network configuration:
 - ```
cd /path/to/stacs/networks/polynet
cp example/polynet.yaml.eg polynet.yaml
cp example/polynet.graph.eg polynet.graph
cp example/polynet.model.eg polynet.model
mkdir records
```
  - This is an 80/20 excitatory/inhibitory network with STDP (from Izhikevich 2006)

# Building and running a network model

- To run a network, we point STACS to the simulation config
  - `./charmrun +p8 ./stacs ./networks/polynet/polynet.yml buildsim`

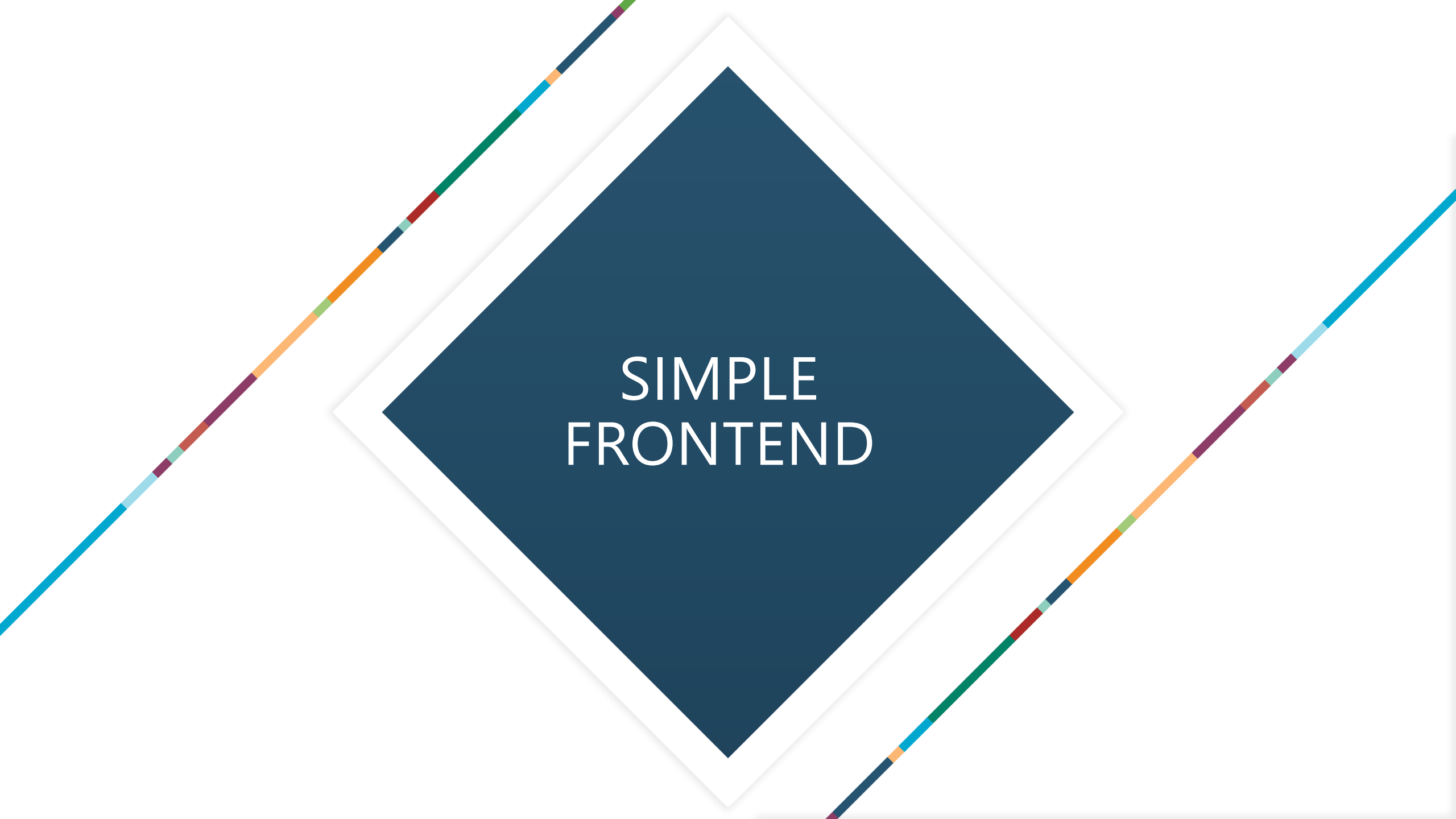
$\underbrace{\text{./charmrun}}_{\text{charm++ launcher}}$   
 $\underbrace{+p8}_{\text{num processes to use}}$

$\underbrace{\text{./stacs}}_{\text{stacs executable}}$

$\underbrace{\text{./networks/polynet/polynet.yml}}_{\text{network simulation config file}}$

$\underbrace{\text{buildsim}}_{\text{runmode}}$
- The run command is composed of a few different components
  - Charm++ launcher (this could also just be on the \$PATH)
  - Number of Charm++ processes to use (e.g. number of network partitions)
  - Optional path to an MPI hostfile (`[-f mpi_hostfile]`, if running on a cluster)
  - Path to the STACS executable (relative to current directory)
  - Path to the network simulation configuration file (relative to current directory)
  - Optional runmode (`buildsim` (`build`, `simulate`), `migrate` / `repart`)
    - Migrate/repart methods require `.part` files that provide new partition indexes (produced from a network partitioner, e.g. ParMETIS)





A central dark blue diamond is surrounded by a white border. Two diagonal lines, one from the top-left to the bottom-right and another from the top-right to the bottom-left, cross the diamond. These lines are composed of small, overlapping colored segments in shades of blue, green, orange, red, and purple.

# SIMPLE FRONTEND

# Structured abstractions for network group organization

- Sango is a compositional spiking neural network domain specific language that is implemented as an internal DSL within Python
- How can we make it easier to develop spiking neural networks?
  - Minimizing boilerplate, and the amount of code that needs to be written
    - Main objective of a domain specific language, using a front-end in Python
  - Focus on developing compositional, hierarchical, and recurrent networks
    - Taking inspiration from Fugu, allowing for procedural generation and code reuse
  - General, extensible, hardware agnostic, interoperable with other tools
    - Focus on a straightforward intermediate representation, easy to map (and debug)
    - Flexibility in neuron and synapse models, leave implementation to the backends
- Available at: <https://github.com/sandialabs/Sango>

# Organize networks similar to filesystem directories

- Organize complex networks analogous to a directory listing of folders (networks themselves), files (topological components), and their contents (nodes and edges)
  - **Network**: the main container that is built out of high-level topological components (including other networks)
  - **NodeGroup**: a group of instantiated nodes sharing the same node model (e.g. a population of neurons)
  - **EdgeGroup**: a group of instantiated edges between two sets of nodes (e.g. a projection of synapses)
- Also provide alias classes (analogous to symlinks) for a layer of indirection and providing reference dependencies for procedural construction of complex network topologies
  - **NodePort**: an alias class pointing to set of (external) nodes (e.g. placeholder nodes for network inputs)
  - **NodeList**: an alias class with a list of (pointers to) nodes (e.g. a collection of nodes for network outputs)

# Constructing networks with simple syntax

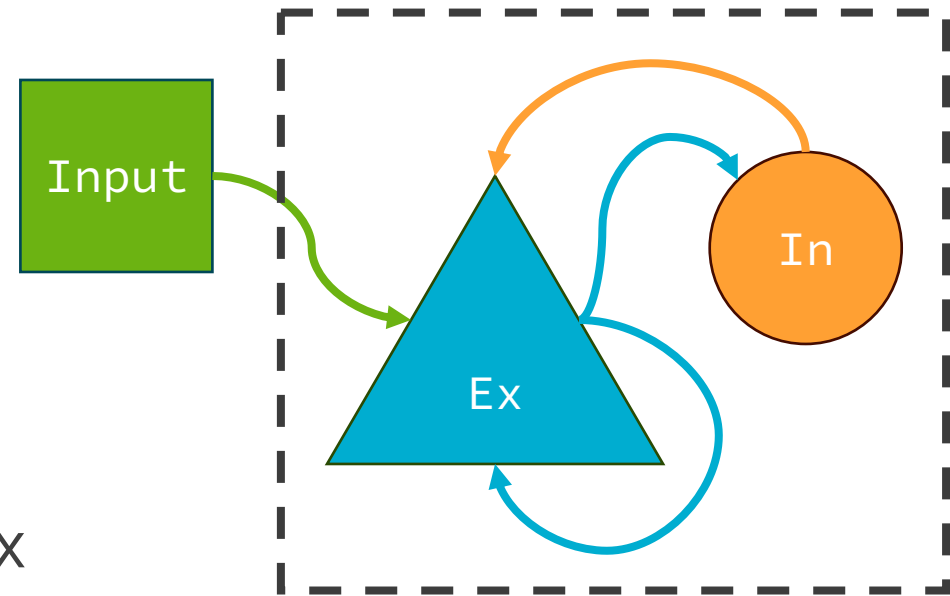
- Network construction through nesting topological components:
  - We can wrap collections of components (including other networks) into the network class, allowing for code reuse and complex hierarchical structures
  - Use simple dot (attributes) and bracket notation (lists), where the combination of names, dots, and brackets determines the “path name” of instantiated nodes (e.g. `net.ff[1].layer[3]`)
- Networks are “built” by iteratively and recursively building any child networks, instantiating any contained topological components (topological sort)
- Two main method definitions:
  - **`__init__()`**: this is where node ports and network parameters are provided
  - **`build()`**: this is where topological components are assigned



PARTITIONING  
EXAMPLE

# Spatially distributed reservoir network

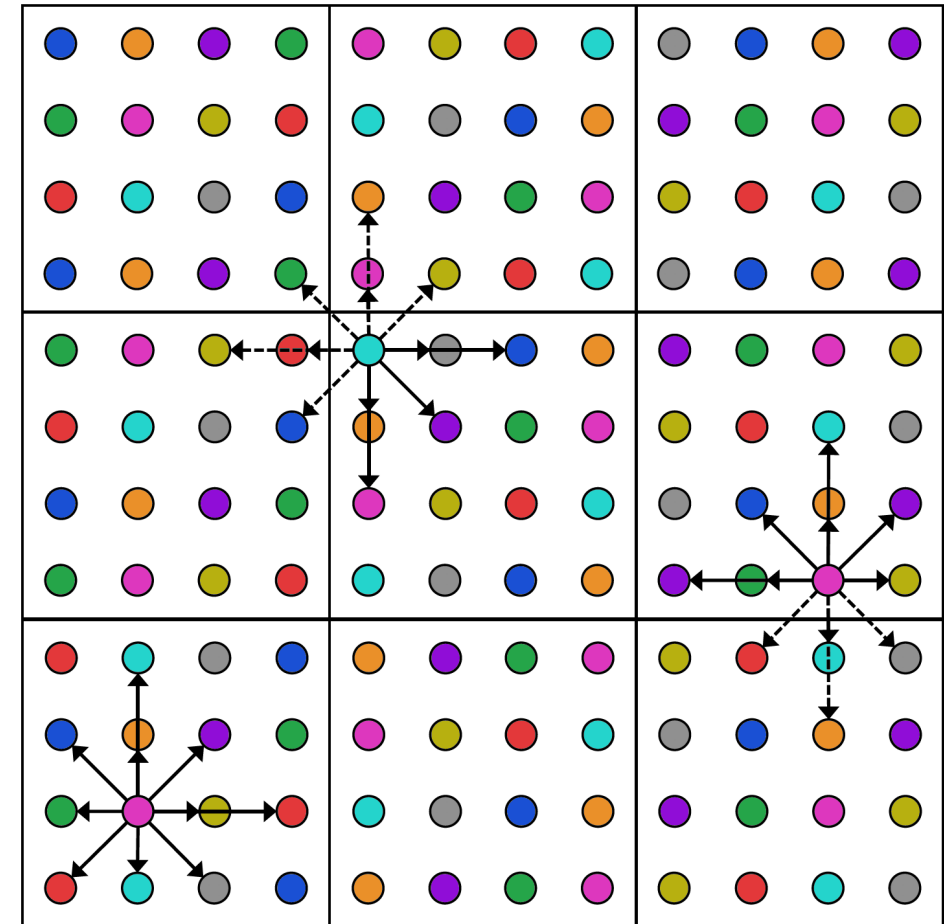
- We will walk through the construction of a simple excitatory-inhibitory reservoir network that is spatially distributed
- 4000 Excitatory Izhikevich neurons
- 1000 Inhibitory Izhikevich neurons
- 1 Hz background input to reservoir
- 100 user-provided input neurons
- Excitatory connections from Input  $\rightarrow$  Ex
- Excitatory connections from Ex  $\rightarrow$  Ex, In
- Inhibitory connections from In  $\rightarrow$  Ex



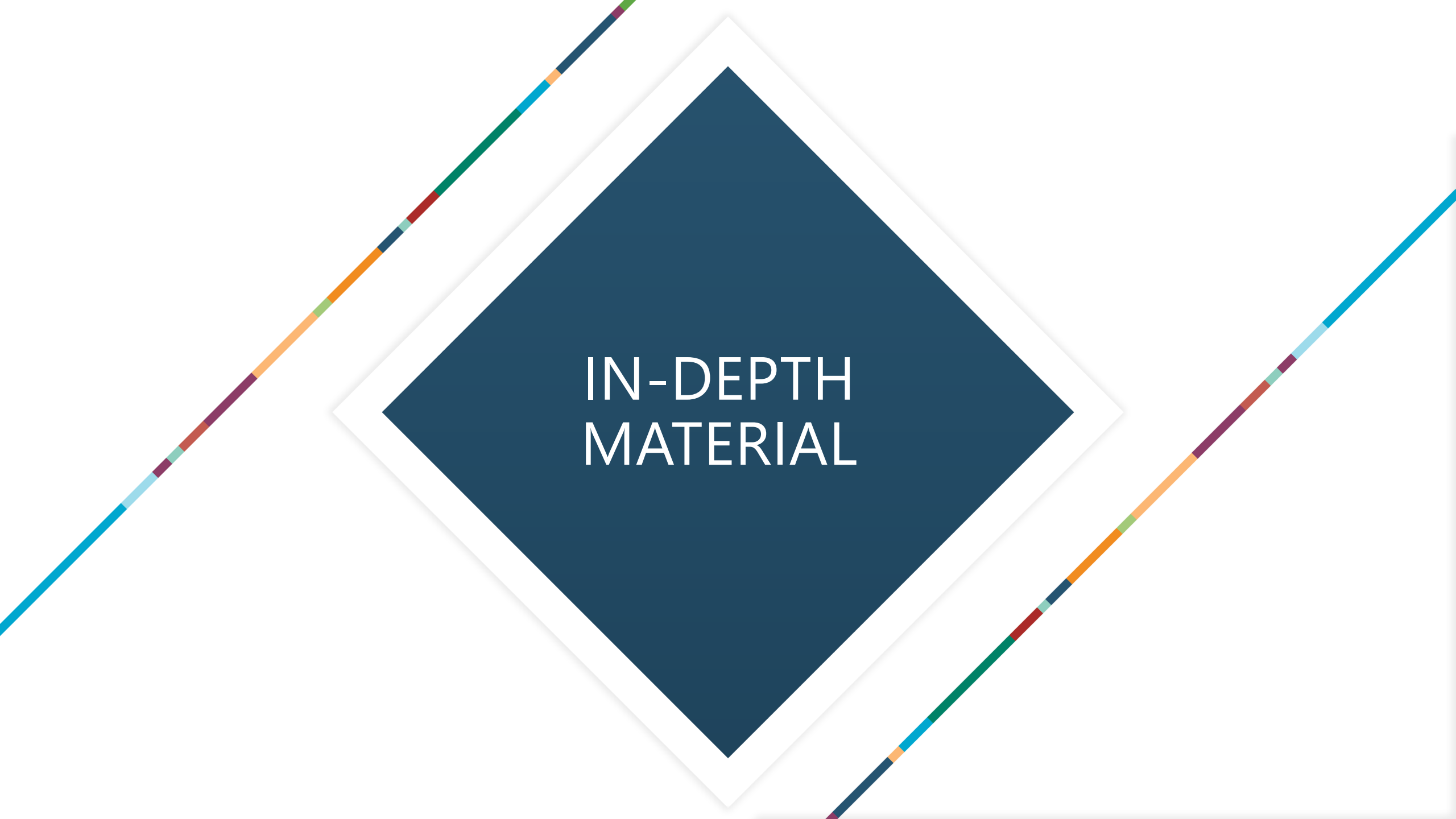
# Spatially partitioning our reservoir network

- Partitioning improves memory locality and reduces communication costs
- The 'edge cut' (the number of edges that go between graph partitions) is a good metric for partitioning
- For a spatially distributed network, a naïve partitioning can work well

For this toy network, going from a 'round-robin' partitioning (by color) to a spatial partitioning (by grid) results in edge cut reduction from 1316 (88.2%) to 448 (32.7%)







IN-DEPTH  
MATERIAL

# Defining a network model

- A network model is composed of three YAML-formatted configuration files:
  - The main config file sets up information related to how the network model will be processed by the simulator (e.g. `network.yml`)
    - Parallelization, total simulation time, random number seed, episodic trials
  - The network graph file defines the connectivity structure of the vertices and edges that compose the network (e.g., `network.graph`)
    - Neuron populations and synaptic projections
  - The network substrate model file parameterizes the time- and event-driven dynamics used by the vertices and edges (e.g., `network.model`)
    - State initialization (potentially from file) during network generation
    - Setting up event watchers and state probes
    - Parameterizing streams which open up ports for I/O

# Defining a network model (vertices and edges)

- Separated into multiple YAML “files” using the using the ``---`` and ``...`` syntax
- Vertices and edges both mostly follow the same model format
- Model names (`modname`) are user-defined strings and are used to differentiate between different parameterizations of substrate dynamics (model names should be unique)
- Model types (`modtype`, the integer identifier) are defined in the model files (located in `stacs/models`) and provide a pointer to the substrate dynamics being used
- Parameters (`param`): information that is related to the model dynamics, but are otherwise shared across all instantiations of a given model type (e.g. the membrane leakage rate for a given neuron population)
  - These are set as simple name-value pairs
- States (`state`): information that are unique per model instantiation and/or are mutable throughout a network simulation (e.g. the membrane voltage for a given neuron)
  - These are set as initialization parameters for network generation (e.g. random number drawn from a uniform interval, bounded normal distribution, etc.)

# Defining a network model (streams, records)

- In addition to vertices and edges, another substrate type is a stream
  - These were originally used to connect a network to external devices through YARP
  - We can also use these to provide inputs from file (e.g. a spike generator)
    - For a simple inputs, the SpikeInput model reads in a YAML-formatted event list
      - These are parameterized as the number of inputs and the path to the file
- For recording more information from a network model, a record “file” can be used:
  - Different event types (applied current stimulus, “clamped” spikes)
  - State probes (defined by the model name, state, and recording frequency)
    - These work for both vertices and edges (indexed by vertex, local edge)

```
type: record
events:
 - spike
probes:
 - tfreq: 1000.0
 modname: exstdp
 state: weight
```

# Defining a network model (graph structure)

- There are two main sections in the network graph file, a list of vertices and a list of edges
  - There is also a section for streams for communication in/out of the network
  - The `modname` references the parameterized substrate models from earlier
- Vertices define groups/populations, and edges define how these are connected
  - The `order` of a vertex entry is how many there are in the group
  - Edge entries have a `source` group and a list of `target` groups
  - Vertex model names and source-target pairs have to be unique
- Vertices allow for spatial instantiation parameters (e.g. within a circle, rectangle)
- Edges are instantiated based on information from the source-target vertices
  - Spatial connectivity parameters (e.g. cutoff distance, periodic boundaries)
  - Indexical connectivity parameters (e.g. local group index, offsets)
  - Connectivity matrix can also be supplied from file (target-major order)
- This system allows for quite a bit of freedom in mixing/matching different substrate models with each other (however, this also includes incompatible models)

# Developing network substrate models

- All the STACS model files are found in `stacs/models`, and are defined as derived classes from the `NetModel` base class (using the `ModelTpl` template, factory method)
  - Network models are represented as a directed graph, and neuron and synapse model dynamics are formulated as stochastic differential equations:
    - $dx = f(x)dt + \sum_{i=1}^n g_i(x)dN_i$  : time-driven computation, event-driven communication
- When defining a network model, its class identifier (the `modtype`) must be supplied
- Within the class constructor, there are multiple named lists for model parameters, model state (for real-valued and "tick" representations), auxiliary states (for allowing an edge model to access and modify vertex state, e.g. synaptic current), and ports
- For the neurons (`vertices`), the dynamics are located in the `Step` function, which are executed at each timestep
- For the synapses (`edges`), the dynamics are located in the `Jump` function, which are executed at discrete event times
- Other functions that may be `Reset` (for episodic trials) and `Leap` (for periodic updates)

# Developing vertex models (step dynamics)

- For neuron models, we define how the states change over time
  - Model equations/dynamics are a function of model parameters and states
  - There is a lot of freedom available in how dynamics are computed
    - There are some random number generators available (uniform, normal)
    - Different numerical integration methods can be used (Euler, Runge-Kutta, etc.)
- Time is provided as the current simulation time `tdrift` and the amount of time left to simulate in the cycle `tdiff`
  - The “tick” representation in STACS is an integer type for precision
  - This can be converted to a real number (float type) through `TICKS_PER_MS`
- Any events (e.g. spikes) that are generated by the model are added to an event list
  - Events are defined as a `diffuse` timestamp, event `type`, and optional payload `data`
  - Helper indexical variables (`source` and `index`) are used for different target rules
    - This is any combination of local (self) and remote (target) vertices and edges

# Developing edge models (jump dynamics)

- For edge models, we define how the states change with discrete event
  - Model equations/dynamics are now also a function of event information, and auxiliary states (corresponding to named states on the target vertex model)
  - There is still a lot of freedom available in how dynamics are computed
- All edge models must have delay set as its first “tick” type state variable
  - This is used to place events into their correct event queues
  - The delay must also be long as the timestep resolution used
- Although STACS mostly follows a point-neuron model, the vertex and (incoming) edge states are collated, which enables potentially interleaved dynamics (e.g. dendrites)
  - The list of vertex states are on index `[0]`, and the `auxstate` provides an indexical pointer to the relevant state in this list
- Information about where the event came from is provided in the `event.source` variable and may be used to select between different state updates (e.g. either side of STDP)
- If time since last spike is required, it will need to be saved in an edge state in the model